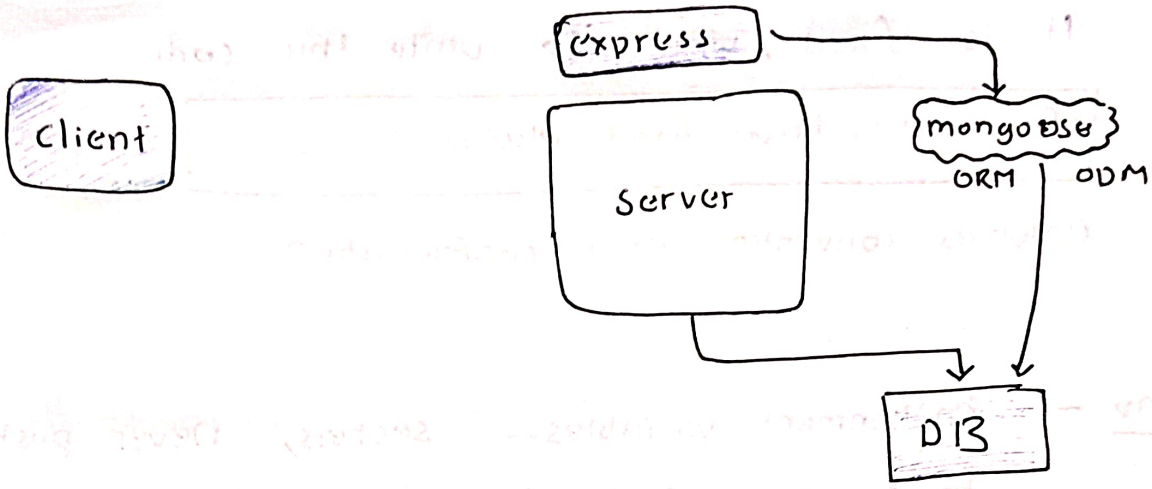
Class-19

March - 21



mongoose : → adds schema, validation and structure to MongoDB

perks

→ cleaner syntax than raw

⇒ (object relational mapping)

ORM : SQL DBs (tables → objects)

ODM : MongoDB (document → object) ← mongoose

⇒ (object-document-mapping)

---

server.js

src

app.js

controller → business logic

routes

models ← structure

utils

endpoints
/reg
/login

firstname
lastname
password
email

db connect

mail

In frameworks, e.g nest JS
Springboot
etc..

It is fixed, where to write the code

- Frameworks have fixed places for code.

enforces convention over configuration

.env → environment variables.. secrets, never push to github. (add to .gitignore)

.env.example → example env file

.gitignore
.env
*.log
node_modules/
.DS_store*

⇒ package : dotenv ... to read environment variables.

usage ..

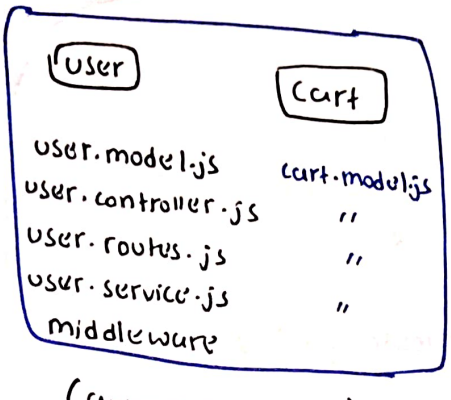
```
cons PORT= process.env.PORT || 5000
```

↑ if no port..

→ In server.js app might come from anywhere..
express, fastapi etc..
we don't care..
we listen on a port.

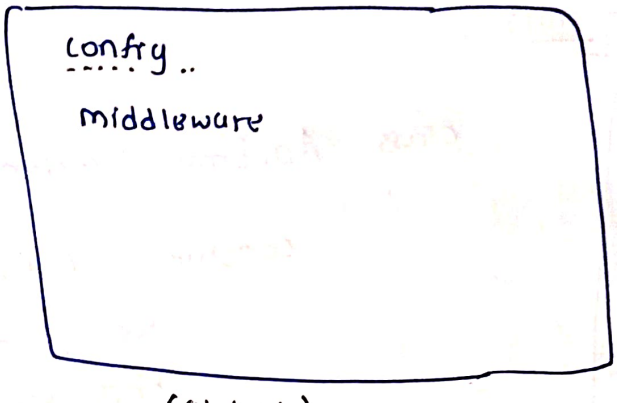
Connecting the DB --

Modules



(group specific)

Common



(global)

// DB connection fails--

// DB is in another continet-- always use try catch..

```
const connectDB = async () => {
    const conn = await mongoose.connect(process.env.URI)
    console.log(`connected ${conn.connection.host}`);
};

export default connectDB
```

Done

Structure ✓

express ✓

Mongoose → connected ✓

Standard structure of response / error

→ Classes are your bestfriend... when you have to do Standarization.

Errors..

```
class ApiError extends Error {
    constructor (statusCode, message) {
        super(message)
        this.statusCode = statusCode;
        this.isOperational = true;
```

→ validation libraries.. ZOD, JOI, ArkType etc..

→ DTO : data transfer object.. (bodyguard) .. its a concept

validation && shape → YES

Data shape

firstName
last name
email
password

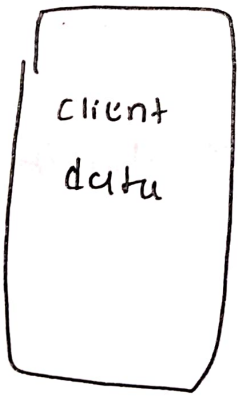
schema

Zod, arkType, express-validator, Joi etc..

Server

base Dto-js.

```
class BaseDto {
    static schema = Joi.object({})
    static validate(data) {
        const { error, value } = this.schema.validate(data, {
            abortEarly: false,
            stripUnknown: true,
        });
        if (error) {
            const errors = error.details.map((d) =>
                d.message);
            return { errors, value: null };
        }
        return { errors: null, value };
    }
}
```



Database design

uuid ,
name
email
password
role
isVerified

verification Token
refresh token
reset Password Token
reset Password Expires

Created At
Updated At

timestamp : true

Code in repo --

/auth.model.js

→ Standard says-- controller me logic -- we don't write.

→ we do it in service.